

[Spark] AI 辅助编写 Spark 开发代码、编写 Spark SQL、解决 Bug、性能调优

辅助资料

[spark_gpt 配套数据.zip](#)

文件名	创建时间	文件类型	文件大小
 orders.txt	2023-05-06 15:45	TXT 文件	1 KB
 story.txt	2023-05-08 14:08	TXT 文件	2 KB
 timestamp.txt	2023-05-06 15:35	TXT 文件	1 KB
 users.txt	2023-05-06 15:45	TXT 文件	1 KB

- `orders.txt` : 模拟的订单数据
- `users.txt` : 模拟的用户数据
- `timestamp.txt` : 模拟的时间戳数据
- `story.txt` : 一个英语短文, 用来做 wordcount

练习题

习题一 编写 WordCount

使用 `story.txt` 数据

你可以尝试用 GPT 来辅助编写一个 WordCount 程序, 要求最后按照各词的频数进行倒序排列并将结果输出到控制台。

习题二 编写 ETL 代码 (时间戳格式统一)

使用 `timestamp.txt` 数据

现在有一个用数据清洗的需求要用 spark 完成。

有一个数据文件里包含了 7 种格式的时间表示

timestamp_millis

yyyy/MM/dd HH:mm:ss.SSS

yyyy-MM-dd HH:mm:ss.SSS

yyyyMMddHHmmss

yyyy-MM-dd' T' HH:mm:ss' +0800'

yyyy-MM-dd' T' HH:mm:ss' Z'

yyyyMMdd' T' HHmmss

示例数据如下：

1649216352000

1621539000000

1594003200000

2023/05/06 15:30:10.123

2021/05/21 10:00:00.000

2020/07/06 14:20:00.500

2023-05-06 15:30:10.123

2021-05-21 10:00:00.000

2020-07-06 14:20:00.500

20230506153010

20210521100000

20200706142000

2023-05-06T15:30:10+0800

2021-05-21T10:00:00+0800

2020-07-06T14:20:00+0800

2023-05-06T15:30:10Z

2021-05-21T10:00:00Z

2020-07-06T14:20:00Z

20230506T153010

20210521T100000

20200706T142000

现在，需要编写一个 spark 程序来把这些时间全部统一成 yyyy-MM-dd HH:mm:ss 格式。你可以尝试使用 GPT 来辅助完成这一任务。

习题三 两数据集 join (Spark SQL)

使用 users.txt 和 orders.txt 数据

现在有一个用户表，一个订单表，两张表的结构大致如下：

users.txt:

user_id	user_name	email	date_of_birth
1	Alice	alice@example.com	1992-06-12
2	Bob	bob@example.com	1985-11-23
3	Carol	carol@example.com	1978-09-14

4	David	david@example.com	1996-02-03
5	Eve	eve@example.com	1990-08-20

orders.txt:

order_id	user_id	product_name	price	order_date
1001	1	Product A	120.0	2023-05-01 12:00:00
1002	1	Product B	75.5	2023-05-02 14:30:00
1003	2	Product C	200.0	2023-05-03 10:45:00
1004	3	Product D	45.0	2023-05-04 16:10:00
1005	4	Product E	250.0	2023-05-05 17:20:00
1006	4	Product F	150.0	2023-05-06 11:30:00
1007	5	Product G	55.0	2023-05-07 13:15:00
1008	3	Product H	100.0	2023-05-08 09:00:00

两张表的数据分别在两个文本文件中，现在你可以尝试用 GPT 写一个 spark 程序来将两张表 join 在一起。最后输出的字段需要有 | user_id | user_name | email | order_id | product_name | price | order_date |

习题四 Spark 作业提交

在使用 spark submit 提交一个 spark 作业时，如何将作业提交到指定的 yarn 队列。（可以试一下用 GPT 和搜索引擎，哪个可以更快地得到答案）

参考答案

习题一 编写 WordCount

第 1 轮提示

提示

帮我用 java maven 写一段 spark 程序，它用一个文件目录来读取文件，并对文件中的内容进行词频统计（英文文章）。

注意，标点符号不算作单词。最后，我希望将统计的结果按照词频倒序排列输出在控制台。

答

首先，确保你已经在你的项目中添加了以下依赖：

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.1.2</version>
  </dependency>
  <dependency>
    <groupId>org.apache.spark</groupId>
```

```
<artifactId>spark-sql_2.12</artifactId>

<version>3.1.2</version>

</dependency>
</dependencies>
```

接下来，以下是一个简单的 Spark 程序，它读取一个文件目录中的所有文件，并计算英文文章中的词频。

```
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.sql.SparkSession;

import java.util.Arrays;
import java.util.List;
import java.util.regex.Pattern;

public class WordCount {

    private static final Pattern SPACE = Pattern.compile(" ");
    private static final Pattern PUNCTUATION = Pattern.compile("[^a-zA-Z0-9 ]");

    public static void main(String[] args) {

        if (args.length < 1) {
            System.err.println("Usage: WordCount <input_directory>");
            System.exit(1);
        }

        String inputDirectory = args[0];

        SparkConf sparkConf = new SparkConf().setAppName("WordCount");
        JavaSparkContext ctx = new JavaSparkContext(sparkConf);
        SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();
```

```

JavaRDD<String> lines = ctx.textFile(inputDirectory);
JavaRDD<String> words = lines.flatMap(line -> {
    String cleanedLine = PUNCTUATION.matcher(line).replaceAll("").toLowerCase();
    return Arrays.asList(SPACE.split(cleanedLine)).iterator();
});

JavaRDD<String> nonEmptyWords = words.filter(word -> !word.isEmpty());
JavaPairRDD<String, Integer> ones = nonEmptyWords.mapToPair(word -> new
Tuple2<>(word, 1));

JavaPairRDD<String, Integer> counts = ones.reduceByKey((Integer i1, Integer i2) ->
i1 + i2);

List<Tuple2<String, Integer>> output = counts.collect();
output.sort((t1, t2) -> -t1._2().compareTo(t2._2()));

for (Tuple2<?, ?> tuple : output) {
    System.out.println(tuple._1() + ": " + tuple._2());
}

spark.stop();
ctx.stop();
}
}

```

这个程序将读取指定目录下的所有文件，将文本内容中的标点符号替换为空格，并将所有文本转换为小写。接着，它将文本拆分为单词，计算每个单词的出现次数，然后按词频对单词进行倒序排列。最后，将排序后的词频统计结果输出到控制台。

反馈

首先，你要有一定的代码阅读能力，知道这段代码首先要求你用命令行的方式传递一个文件路径，如果说命令行参数没有传递的话，这个程序就会直接报错退出。

```

public static void main(String[] args) {

    if (args.length < 1) {
        System.err.println("Usage: WordCount <input_directory>");
        System.exit(1);
    }
}

```

另外，这段代码运行后会先抛出一个异常。

```

23/05/08 14:31:08 INFO SparkContext: Successfully stopped SparkContext
Exception in thread "main" org.apache.spark.SparkException: Create breakpoint: A master URL must be set in your configuration
    at org.apache.spark.SparkContext.<init>(SparkContext.scala:394)
    at org.apache.spark.api.java.JavaSparkContext.<init>(JavaSparkContext.scala:58)
    at org.example.WordCount.main(WordCount.java:30)

```

你可以把这个异常反馈给 GPT 让它继续迭代，不过这种简单的异常一般我们手动就能解决，这个异常是本地运行 Spark 时，Spark 配置里没写 master 的配置导致的

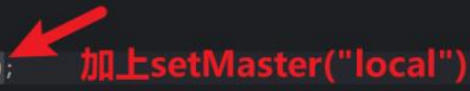
。

```

String inputDirectory = args[0];

SparkConf sparkConf = new SparkConf().setAppName("WordCount");
JavaSparkContext ctx = new JavaSparkContext(sparkConf);
SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();

```



现在，我们可以再次尝试运行程序，这个时候还会再报一个错：

```

23/05/08 14:45:34 INFO DAGScheduler: Job 0 finished: collect at WordCount.java:43, t
Exception in thread "main" java.lang.UnsupportedOperationException: Create breakpoint
    at java.util.AbstractList.set(AbstractList.java:132)
    at java.util.AbstractList$ListItr.set(AbstractList.java:426)
    at java.util.List.sort(List.java:482)
    at org.example.WordCount.main(WordCount.java:44)

```

对于编程经验不太丰富的程序员来说，遇到这种异常可能就要头疼一阵子了，这个时候，我们可以让 GPT 来修复这个异常。

第 2 轮提示

提示

报错

```
Exception in thread "main" java.lang.UnsupportedOperationException  
  
at java.util.AbstractList.set(AbstractList.java:132)  
  
at java.util.AbstractList$ListItr.set(AbstractList.java:426)  
  
at java.util.List.sort(List.java:482)  
  
at org.example.WordCount.main(WordCount.java:43)
```

答

抱歉给你带来的不便。这个错误是因为 `collect()` 方法返回的 `List` 实例是不可修改的。我们需要创建一个新的 `ArrayList`，将结果复制进去，然后再进行排序。请将以下修改后的代码替换到之前的代码中：

```
import org.apache.spark.SparkConf;  
import org.apache.spark.api.java.JavaPairRDD;  
import org.apache.spark.api.java.JavaRDD;  
import org.apache.spark.api.java.JavaSparkContext;  
import org.apache.spark.sql.SparkSession;  
import scala.Tuple2;  
  
import java.util.ArrayList;  
import java.util.Arrays;  
import java.util.List;  
import java.util.regex.Pattern;  
  
public class WordCount {
```

```

private static final Pattern SPACE = Pattern.compile(" ");
private static final Pattern PUNCTUATION = Pattern.compile("[^a-zA-Z0-9 ]");

public static void main(String[] args) {

    if (args.length < 1) {
        System.err.println("Usage: WordCount <input_directory>");
        System.exit(1);
    }

    String inputDirectory = args[0];

    SparkConf sparkConf = new SparkConf().setAppName("WordCount");
    JavaSparkContext ctx = new JavaSparkContext(sparkConf);
    SparkSession spark = SparkSession.builder().config(sparkConf).getOrCreate();

    JavaRDD<String> lines = ctx.textFile(inputDirectory);
    JavaRDD<String> words = lines.flatMap(line -> {
        String cleanedLine = PUNCTUATION.matcher(line).replaceAll("").toLowerCase();
        return Arrays.asList(SPACE.split(cleanedLine)).iterator();
    });

    JavaRDD<String> nonEmptyWords = words.filter(word -> !word.isEmpty());
    JavaPairRDD<String, Integer> ones = nonEmptyWords.mapToPair(word -> new
Tuple2<>(word, 1));
    JavaPairRDD<String, Integer> counts = ones.reduceByKey((Integer i1, Integer i2) ->
i1 + i2);

    List<Tuple2<String, Integer>> output = counts.collect();
    List<Tuple2<String, Integer>> sortedOutput = new ArrayList<>(output);
    sortedOutput.sort((t1, t2) -> -t1._2().compareTo(t2._2()));

    for (Tuple2<?, ?> tuple : sortedOutput) {
        System.out.println(tuple._1() + ": " + tuple._2());
    }
}

```

```
        spark.stop();  
        ctx.stop();  
    }  
}
```

这个修改后的代码在创建一个新的 `ArrayList`，将 `collect()` 的结果复制到新列表中，然后在新列表上进行排序。这样就不会再遇到不支持的操作异常。

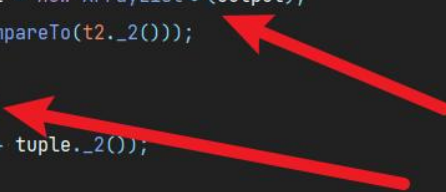
反馈

这个地方我们可以先把代码复制到 IDEA 里测试一下效果。

```
23/05/08 15:31:30 INFO TaskSchedulerImpl: Killing all running tasks in stage 1: Stage finished  
23/05/08 15:31:30 INFO DAGScheduler: Job 0 finished: collect at WordCount.java:43, took 0.578391 s  
the: 22  
and: 19  
their: 11  
of: 10  
to: 9  
he: 8  
a: 7  
johnson: 6  
his: 6  
mr: 6  
in: 5  
children: 5  
as: 4  
village: 4  
with: 3  
young: 2
```

可以看到，现在程序已经成功地输出了我们想要的结果。而且 GPT 预测出了问题的原因，并给出了解决方案。下面就是 GPT 做出的主要修改。最后要排序的时候，将原先 `list` 中的数据复制到了一份新的 `ArrayList` 中。

```

1000 List


```

习题二 编写 ETL 代码（时间戳格式统一）

第 1 轮提示

提示

我现在有一个文件，它里的数据格式如下：

1649216352000

1621539000000

1594003200000

2023/05/06 15:30:10.123

2021/05/21 10:00:00.000

2020/07/06 14:20:00.500

2023-05-06 15:30:10.123

2021-05-21 10:00:00.000

2020-07-06 14:20:00.500

... ..

它里面包含 7 种时间格式

timestamp_millis

yyyy/MM/dd HH:mm:ss.SSS

yyyy-MM-dd HH:mm:ss.SSS

yyyyMMddHHmmss

yyyy-MM-dd' T' HH:mm:ss' +0800'

yyyy-MM-dd' T' HH:mm:ss' Z'

yyyyMMdd' T' HHmmss

我现在想用 java 和 maven 写一个 spark 程序，读取文件后将里面的时间全部改为 yyyy-MM-dd HH:mm:ss，记得处理时区问题。

答

要实现这个功能，首先需要创建一个 Maven 项目，然后在项目中添加 Apache Spark 和 Java 的相关依赖。接下来，创建一个 Java 类，使用 Spark 读取文件，处理不同的时间格式，转换为统一的格式，并处理时区问题。

1. 创建一个 Maven 项目，并在 **pom.xml** 中添加以下依赖：

```
<dependencies>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-core_2.12</artifactId>
    <version>3.2.0</version>
  </dependency>
  <dependency>
    <groupId>org.apache.spark</groupId>
    <artifactId>spark-sql_2.12</artifactId>
    <version>3.2.0</version>
  </dependency>
</dependencies>
```

2. 在项目中创建一个名为 **TimeFormatConverter** 的 Java 类，实现如下功能：

- 使用 Spark 读取文件
- 定义一个方法 **convertTimeFormat**，用于将不同的时间格式转换为“yyyy-MM-dd HH:mm:ss”格式
- 处理时区问题

```
import org.apache.spark.sql.*;
import org.apache.spark.sql.types.*;
import java.time.*;
import java.time.format.*;
import java.util.Arrays;
import java.util.List;

public class TimeFormatConverter {
    public static void main(String[] args) {
        // 创建 SparkSession
        SparkSession spark = SparkSession.builder()
            .appName("Time Format Converter")
            .master("local")
            .getOrCreate();

        // 读取文件
        Dataset<String> inputDataset =
spark.read().textFile("path/to/your/input/file.txt");

        // 定义 Schema
        StructType schema = new StructType().add("timestamp", DataTypes.StringType);

        // 转换为 DataFrame
        Dataset<Row> inputDF = inputDataset.withColumn("timestamp",
inputDataset.col("value")).drop("value");

        // 定义时间格式转换 UDF
```

```

        spark.udf().register("convertTimeFormat", (String timestamp) ->
convertTimeFormat(timestamp), DataTypes.StringType);

// 应用时间格式转换 UDF
Dataset<Row> outputDF = inputDF.withColumn("converted_timestamp",
functions.callUDF("convertTimeFormat", inputDF.col("timestamp")));

// 输出结果
outputDF.show(false);

// 停止 SparkSession
spark.stop();
}

private static String convertTimeFormat(String timestamp) {
    List<DateTimeFormatter> formatters = Arrays.asList(
        DateTimeFormatter.ofPattern("yyyyMMddHHmmss"),
        DateTimeFormatter.ofPattern("yyyy/MM/dd HH:mm:ss.SSS"),
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss.SSS"),
        DateTimeFormatter.ofPattern("yyyy-MM-dd' T' HH:mm:ss' +0800' "),
        DateTimeFormatter.ofPattern("yyyy-MM-dd' T' HH:mm:ss' Z' "),
        DateTimeFormatter.ofPattern("yyyyMMdd' T' HHmmss")
    );

    for (DateTimeFormatter formatter : formatters) {
        try {
            LocalDateTime dateTime = LocalDateTime.parse(timestamp, formatter);
            ZoneId sourceZone = ZoneId.systemDefault();
            ZoneId targetZone = ZoneId.of("UTC");
            ZonedDateTime sourceZonedDateTime = dateTime.atZone(sourceZone);
            ZonedDateTime targetZonedDateTime =
sourceZonedDateTime.withZoneSameInstant(targetZone);

            return targetZonedDateTime.format(DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss"));
        } catch (DateTimeParseException e) {
            // Ignore the exception and try

```

反馈

可以看到，GPT 在尝试使用 DataFrame + UDF 的方式来完成。不过代码没写完，我们可以让它继续。

第 2 轮提示

提示

继续

答

```
        the next formatter
    }
}

// 如果输入的时间戳是毫秒级别的
try {
    long millis = Long.parseLong(timestamp);
    Instant instant = Instant.ofEpochMilli(millis);
    ZoneId sourceZone = ZoneId.systemDefault();
    ZoneId targetZone = ZoneId.of("UTC");
    ZonedDateTime sourceZonedDateTime = instant.atZone(sourceZone);
    ZonedDateTime targetZonedDateTime =
sourceZonedDateTime.withZoneSameInstant(targetZone);

    return targetZonedDateTime.format(DateTimeFormatter.ofPattern("yyyy-MM-dd
HH:mm:ss"));
} catch (NumberFormatException e) {
    // Ignore the exception
}

// 如果无法识别时间格式，返回原始字符串
```

```
        return timestamp;
    }
}
```

这个 Java 类会读取文件中的每一行，尝试使用预定义的时间格式解析字符串。如果解析成功，则将其转换为统一的“yyyy-MM-dd HH:mm:ss”格式，并处理时区问题。如果无法解析，将保留原始字符串。

运行这个 Java 程序后，它将读取文件并将不同时间格式的时间戳转换为“yyyy-MM-dd HH:mm:ss”格式，同时处理时区问题。注意替换`path/to/your/input/file.txt`为你实际文件的路径。

反馈

现在 GPT 已经有模有样地帮我们把代码写完了，这个时候我们可以把它粘出来测试一下。

下面就是最终的运行结果，可以看到一步到位跑出了结果。

```

+-----+-----+
|timestamp|converted_timestamp|
+-----+-----+
|1649216352000|2022-04-06 03:39:12|
|1621539000000|2021-05-20 19:30:00|
|1594003200000|2020-07-06 02:40:00|
|2023/05/06 15:30:10.123|2023-05-06 07:30:10|
|2021/05/21 10:00:00.000|2021-05-21 02:00:00|
|2020/07/06 14:20:00.500|2020-07-06 06:20:00|
|2023-05-06 15:30:10.123|2023-05-06 07:30:10|
|2021-05-21 10:00:00.000|2021-05-21 02:00:00|
|2020-07-06 14:20:00.500|2020-07-06 06:20:00|
|20230506153010|2023-05-06 07:30:10|
|20210521100000|2021-05-21 02:00:00|
|20200706142000|2020-07-06 06:20:00|
|2023-05-06T15:30:10+0800|2023-05-06 07:30:10|
|2021-05-21T10:00:00+0800|2021-05-21 02:00:00|
|2020-07-06T14:20:00+0800|2020-07-06 06:20:00|
|2023-05-06T15:30:10Z|2023-05-06 07:30:10|
|2021-05-21T10:00:00Z|2021-05-21 02:00:00|
|2020-07-06T14:20:00Z|2020-07-06 06:20:00|
|20230506T153010|2023-05-06 07:30:10|
|20210521T100000|2021-05-21 02:00:00|
+-----+-----+

```

习题三 两数据集 join (sql)

第 1 轮提示

提示

我现在有两张表

用户表里有下述字段：

user_id

user_name

email

data_of_birth

订单表里有下述字段：

order_id

user_id

product_name

price

order_date

两张表分别在两个 csv 文件里，分隔符是\t

帮我用 java maven 写一个 spark 程序根据两个文件路径把表读进来并根据 user_id 进行 join。join 后输出的字段要包括

| user_id | user_name | email | order_id | product_name | price | order_date |

将 join 后的结果显示在控制台

答

首先，需要创建一个 Java Maven 项目并添加 Spark 依赖。创建一个新的 Java 类（如：SparkJoinExample.java），然后在该类中编写如下代码：

```
import org.apache.spark.SparkConf;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.api.java.JavaSparkContext;
import org.apache.spark.api.java.function.Function;
import org.apache.spark.sql.Dataset;
import org.apache.spark.sql.Row;
import org.apache.spark.sql.Session;

import java.util.Arrays;

public class SparkJoinExample {
    public static void main(String[] args) {
        if (args.length != 2) {
            System.err.println("Usage: SparkJoinExample <user_file> <order_file>");
        }
    }
}
```

```

        System.exit(1);
    }

    String userFile = args[0];
    String orderFile = args[1];

    SparkConf conf = new SparkConf().setAppName("Spark Join Example");
    JavaSparkContext sc = new JavaSparkContext(conf);
    SparkSession spark = SparkSession.builder().config(conf).getOrCreate();

    JavaRDD<String> userFileLines = sc.textFile(userFile);
    JavaRDD<String> orderFileLines = sc.textFile(orderFile);

    // Read and process user data
    JavaRDD<User> userRDD = userFileLines.map(new Function<String, User>() {
        @Override
        public User call(String line) throws Exception {
            String[] parts = line.split("\t");
            User user = new User(Integer.parseInt(parts[0]), parts[1], parts[2],
parts[3]);
            return user;
        }
    });

    // Read and process order data
    JavaRDD<Order> orderRDD = orderFileLines.map(new Function<String, Order>() {
        @Override
        public Order call(String line) throws Exception {
            String[] parts = line.split("\t");
            Order order = new Order(Integer.parseInt(parts[0]),
Integer.parseInt(parts[1]), parts[2], Double.parseDouble(parts[3]), parts[4]);
            return order;
        }
    });

    Dataset<Row> userDF = spark.createDataFrame(userRDD, User.class);

```

```

    Dataset<Row> orderDF = spark.createDataFrame(orderRDD, Order.class);

    // Join the dataframes on user_id
    Dataset<Row> joinedDF = userDF.join(orderDF, "user_id");

    // Select the required columns
    Dataset<Row> selectedDF = joinedDF.select("user_id", "user_name", "email",
"order_id", "product_name", "price", "order_date");

    // Show the result
    selectedDF.show();

    sc.stop();
}
}

```

接下来，需要为 User 和 Order 类创建两个新的 Java 类，如下所示：

```

// User.java
import java.io.Serializable;

public class User implements Serializable {
    private int user_id;
    private String user_name;
    private String email;
    private String date_of_birth;

    public User(int user_id, String user_name, String email, String date_of_birth) {
        this.user_id = user_id;
        this.user_name = user_name;
        this.email = email;
        this.date_of_birth = date_of_birth;
    }

    // Getters and setters

```

```
}

// Order.java
import java.io.Serializable;

public class Order implements Serializable {
    private int order_id;
    private int user_id;
    private String product_name;
    private double price;
    private String order_date;

    public Order(int order_id, int user_id, String product_name, double price, String
order_date) {
        this.order_id = order_id;
        this.user_id = user_id
```

反馈

代码没有写完，让 GPT 继续

第 2 轮提示

提示

继续

答

```
        this.product_name = product_name;
        this.price = price;
        this.order_date = order_date;
    }
```

```
// Getters and setters  
}
```

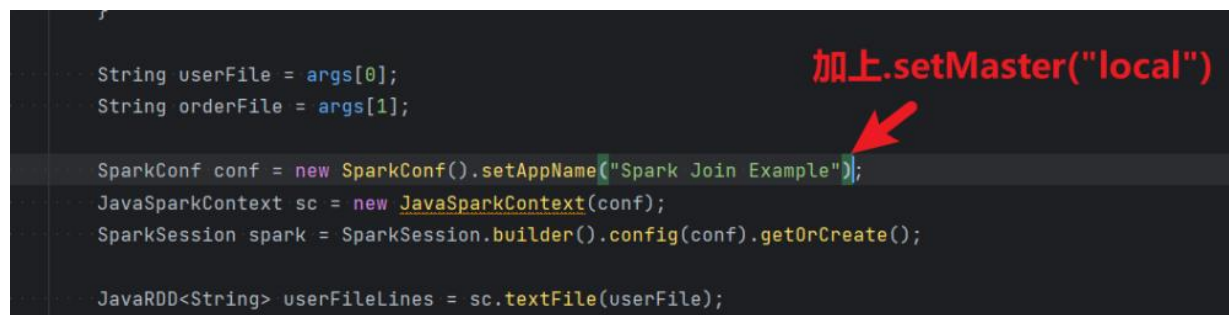
现在，你可以使用以下命令在命令行中运行该程序（确保已将项目打包为 JAR 文件）：

```
spark-submit --class SparkJoinExample --master local[4] target/spark-join-example-1.0-SNAPSHOT.jar /path/to/user/file.tsv /path/to/order/file.tsv
```

其中，`/path/to/user/file.tsv` 和 `/path/to/order/file.tsv` 是用户表和订单表文件的路径。运行该程序后，你将在控制台看到 join 后的结果。

反馈

现在，我们可以把代码复制到 IDEA 中进行测试。记得注意，GPT 给我们的代码是在命令行里去指定 master 的，我们也可以在代码里把 master 写死。



The screenshot shows a code editor with the following Java code:

```
String userFile = args[0];  
String orderFile = args[1];  
  
SparkConf conf = new SparkConf().setAppName("Spark Join Example");  
JavaSparkContext sc = new JavaSparkContext(conf);  
SparkSession spark = SparkSession.builder().config(conf).getOrCreate();  
  
JavaRDD<String> userFileLines = sc.textFile(userFile);
```

A red arrow points to the `setAppName` method call, with the text **加上.setMaster("local")** written in red above it.

现在，程序运行会报下面的异常：

```

java.lang.NumberFormatException: Create breakpoint : For input string: "order_id"
— at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
— at java.lang.Integer.parseInt(Integer.java:580)
— at java.lang.Integer.parseInt(Integer.java:615)
— at org.example.test2.SparkJoinExample$2.call(SparkJoinExample.java:45)
— at org.example.test2.SparkJoinExample$2.call(SparkJoinExample.java:41)
— at org.apache.spark.api.java.JavaPairRDD$.anonfun$toScalaFunction$1(JavaPairRDD.scala:1070)
— at scala.collection.Iterator$$anon$10.next(Iterator.scala:461)
— at scala.collection.Iterator$$anon$10.next(Iterator.scala:461)
— at org.apache.spark.sql.catalyst.expressions.GeneratedClass$GeneratedIteratorForCodegenStage2.processNext(Unknown Source)
— at org.apache.spark.sql.execution.BufferedRowIterator.hasNext(BufferedRowIterator.java:43)
— at org.apache.spark.sql.execution.WholeStageCodegenExec$$anon$1.hasNext(WholeStageCodegenExec.scala:759)
— at scala.collection.Iterator$$anon$10.hasNext(Iterator.scala:460)
— at org.apache.spark.shuffle.sort.BypassMergeSortShuffleWriter.write(BypassMergeSortShuffleWriter.java:140)
— at org.apache.spark.shuffle.ShuffleWriteProcessor.write(ShuffleWriteProcessor.scala:59)
— at org.apache.spark.scheduler.ShuffleMapTask.runTask(ShuffleMapTask.scala:99)
— at org.apache.spark.scheduler.ShuffleMapTask.runTask(ShuffleMapTask.scala:52)
— at org.apache.spark.scheduler.Task.run(Task.scala:131)
— at org.apache.spark.executor.Executor$TaskRunner.$anonfun$run$3(Executor.scala:506)
— at org.apache.spark.util.Utils$.tryWithSafeFinally(Utils.scala:1462)
> — at org.apache.spark.executor.Executor$TaskRunner.run(Executor.scala:509) <3 internal lines>

```

我们可以回顾 GPT 写的代码逻辑，其实它是当做 csv 文件中没有标题行的。这倒是标题 `order_id` 作为字符串没有办法解析成 `int`，从而报错。你可以自己手动修复这个异常，也可以继续让 GPT 继续修正。

第 3 轮提示

提示

报错

```
java.lang.NumberFormatException: For input string: "order_id"
```

答

这个错误可能是由于文件中的表头（列名）没有被过滤掉导致的。我们需要在处理数据之前过滤掉表头。在 `userFileLines` 和 `orderFileLines` 之后添加以下过滤代码：

```

// Filter out the header lines
String userHeader = userFileLines.first();
userFileLines = userFileLines.filter(line -> !line.equals(userHeader));

String orderHeader = orderFileLines.first();
orderFileLines = orderFileLines.filter(line -> !line.equals(orderHeader));

```

将上述代码添加到 **main** 方法中，如下所示：

```
public class SparkJoinExample {  
    public static void main(String[] args) {  
        if (args.length != 2) {  
            System.err.println("Usage: SparkJoinExample <user_file> <order_file>");  
            System.exit(1);  
        }  
  
        String userFile = args[0];  
        String orderFile = args[1];  
  
        SparkConf conf = new SparkConf().setAppName("Spark Join Example");  
        JavaSparkContext sc = new JavaSparkContext(conf);  
        SparkSession spark = SparkSession.builder().config(conf).getOrCreate();  
  
        JavaRDD<String> userFileLines = sc.textFile(userFile);  
        JavaRDD<String> orderFileLines = sc.textFile(orderFile);  
  
        // Filter out the header lines  
        String userHeader = userFileLines.first();  
        userFileLines = userFileLines.filter(line -> !line.equals(userHeader));  
  
        String orderHeader = orderFileLines.first();  
        orderFileLines = orderFileLines.filter(line -> !line.equals(orderHeader));  
  
        // ... rest of the code ...  
    }  
}
```

现在重新运行程序，**NumberFormatException** 问题应该已经解决。

反馈

现在 GPT 的思路是将 RDD 里的第一行用 `first()` 取出来，然后对 rdd 里的数据集进行 `filter` 过滤。按照 GPT 的建议对代码进行修改，最后发现代码成功跑通。

```
23/05/08 16:19:47 INFO DAGScheduler: Job 4 finished: show at SparkJoinExample.java:66, took 0.138558 s
23/05/08 16:19:47 INFO CodeGenerator: Code generated in 10.1305 ms

+-----+-----+-----+-----+-----+-----+
|user_id|user_name|email|order_id|product_name|price|order_date|
+-----+-----+-----+-----+-----+-----+
|1|Alice|alice@example.com|1001|Product A|120.0|2023-05-01 12:00:00|
|1|Alice|alice@example.com|1002|Product B|75.5|2023-05-02 14:30:00|
|2|Bob|bob@example.com|1003|Product C|200.0|2023-05-03 10:45:00|
|3|Carol|carol@example.com|1004|Product D|45.0|2023-05-04 16:10:00|
|3|Carol|carol@example.com|1008|Product H|100.0|2023-05-08 09:00:00|
|4|David|david@example.com|1005|Product E|250.0|2023-05-05 17:20:00|
|4|David|david@example.com|1006|Product F|150.0|2023-05-06 11:30:00|
|5|Eve|eve@example.com|1007|Product G|55.0|2023-05-07 13:15:00|
+-----+-----+-----+-----+-----+-----+
```

习题四 Spark 作业提交

第 1 轮提示

提示

`spark submit` 命令提交 spark 作业时如何指定要提交到 yarn 的哪个队列

答

要将 Spark 作业提交到 YARN 的特定队列，可以在 `spark-submit` 命令中使用 `--conf` 选项设置 `spark.yarn.queue` 属性。以下是一个示例：

```
spark-submit \
  --class YourMainClass \
  --master yarn \
  --deploy-mode cluster \
  --conf spark.yarn.queue=your_queue_name \
  /path/to/your/application.jar \
```

```
arg1 arg2 ...
```

在这个例子中，将 `your_queue_name` 替换为你想要提交的 YARN 队列名称，将 `/path/to/your/application.jar` 替换为你的应用程序 JAR 文件的路径，将 `YourMainClass` 替换为包含 `main` 方法的 Java 类的名称，将 `arg1 arg2 ...` 替换为你的程序所需的任何命令行参数。

这将提交一个 Spark 作业到 YARN 集群的指定队列。

反馈

已经得到正确答案了，下面是官方文档的截图：

<code>spark.yarn.am.memoryOverhead</code>	AM memory * 0.10, with minimum of 384	Same as <code>spark.driver.memoryOverhead</code> , but for the YARN Application Master in client mode.
<code>spark.yarn.queue</code>	default	The name of the YARN queue to which the application is submitted.